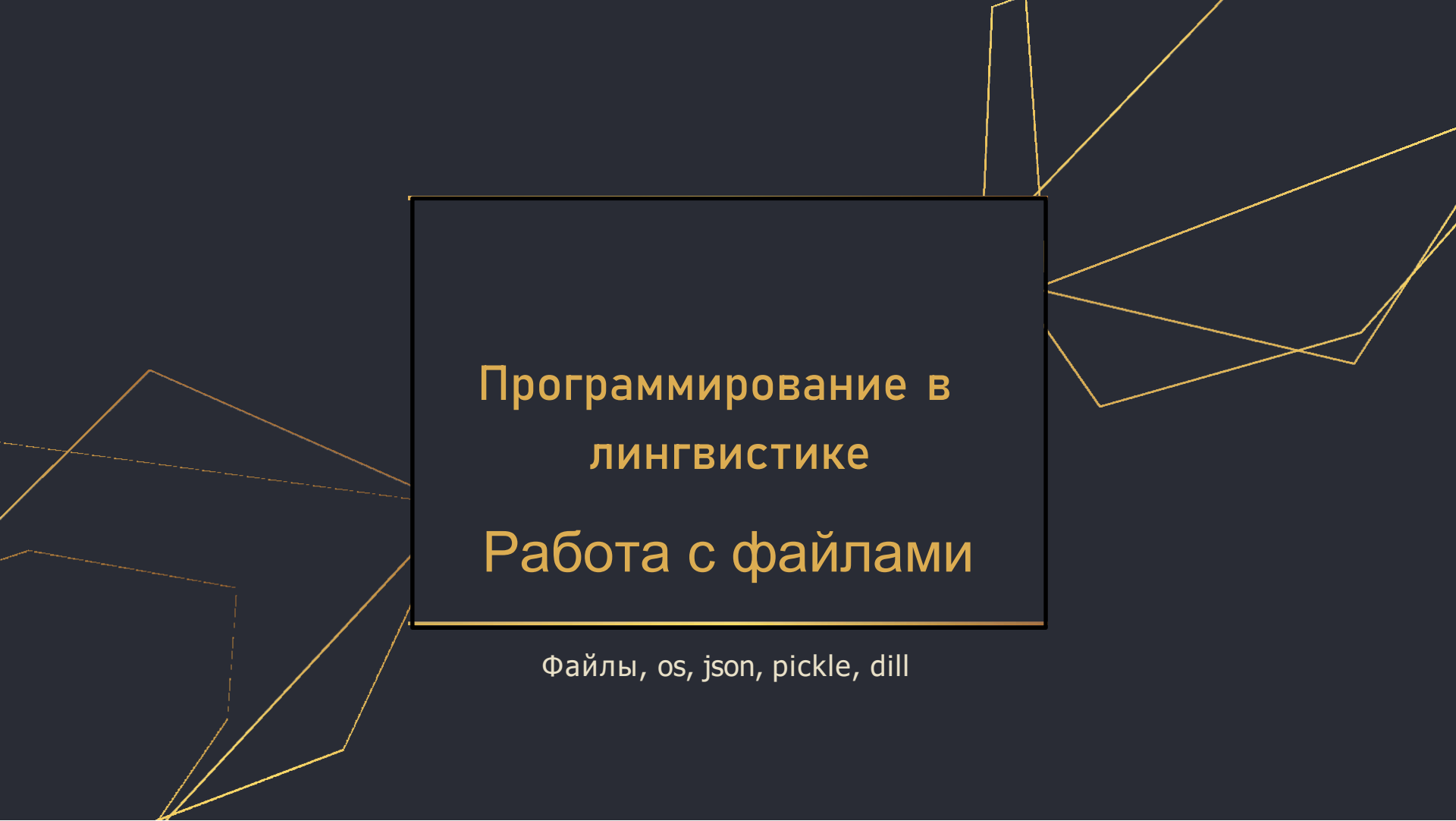


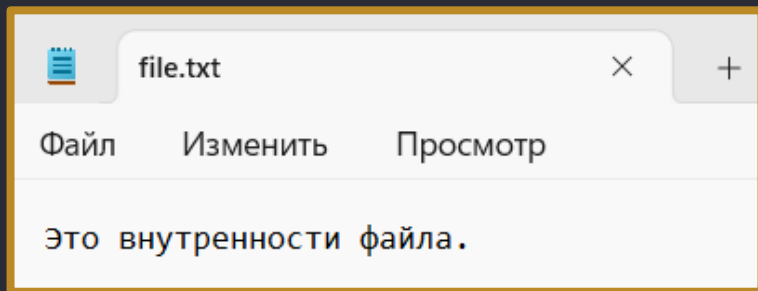
The background of the slide features several thin, light-yellow lines that form abstract, angular shapes, resembling stylized mountains or a network diagram. These lines are scattered across the dark blue background, with some lines intersecting to form small polygons.

# Программирование в лингвистике

## Работа с файлами

Файлы, os, json, pickle, dill

# Файлы в Python



```
fin = open("file.txt", "r")
read_text = fin.read()
print(read_text)
fin.close()
```

Это внутренности файла.

Python работает с файлами через систему.

Для работы с файлами у Python есть **дескриптор файла** – специальный класс, который позволяет **читать данные из файла**.

В мире Python изначально существует два типа файлов: **текстовые** (хранятся в формате .txt, .rtf) и **бинарные** (хранятся в формате .bin).

# Открытие файла

```
file = open(path, mode, encoding)
```

> **path** – абсолютный или относительный путь к файлу.

> **mode** – режим работы с файлом, str. Может быть 'r' – read, 'w' – write, 'rb' – read bin, 'wb' – write bin, 'r+' – read & write, 'rb+' – read & write bin и др.

> **encoding** – кодировка файла, str. Часто следует указывать utf8.

Прежде чем начать работать с файлом, следует его **открыть**.

Функция **open()** создает объект класса **IOWrapper**, у которого есть свои методы чтения, записи и закрытия. **Path** - это путь к файлу, единственный обязательный аргумент.

```
fin = open("file.txt", "r", encoding="utf8")
```

# Особенности path

```
open('C:\train\new\text.txt')
```



```
open('C:\\train\\new\\text.txt')
```

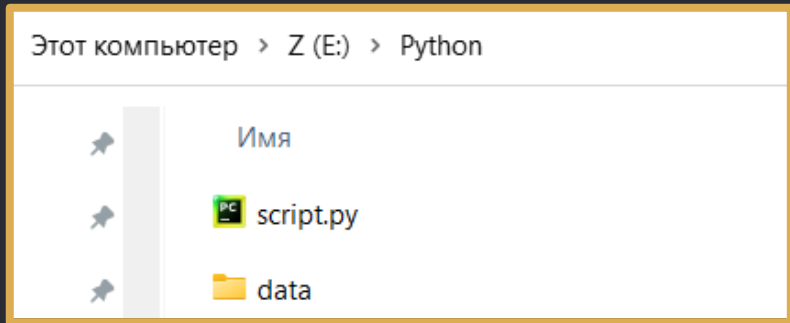


```
open(r'C:\train\new\text.txt')
```



В Windows в путях используется бэкслэш. Пути – строки, поэтому Python будет искать специальные символы вроде `\n`. Так, можно либо экранировать бэкслэш, либо ставить флаг `r`. В Linux и MacOS используется обычный слэш.

# Особенности path



Здесь пути к файлу `text.txt`, находящемуся в папке `data`, для кода `script.py`, будут такими:

**Абсолютный** - `E:\Python\data\text.txt`

**Относительный** – `data\text.txt`

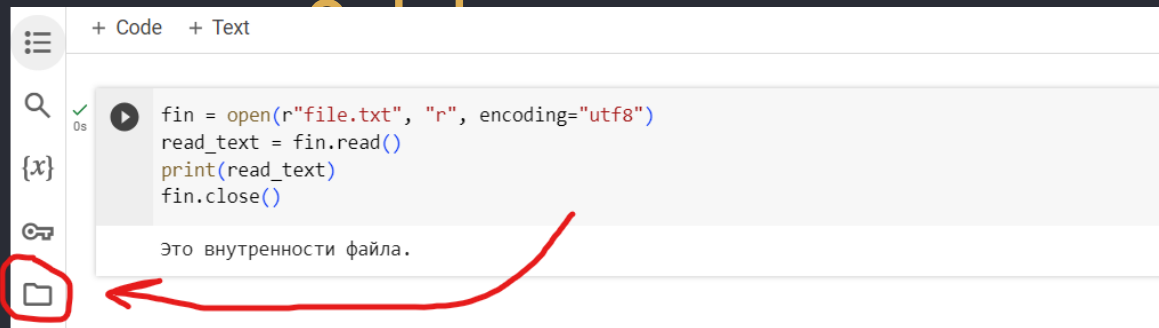
Если файл лежит прямо в той директории, где находится скрипт, то достаточно будет `open('text.txt')`.

Пути бывают **абсолютными** и **относительными**. Абсолютный путь включает в себя **всё от корня** (буквы диска в Windows, папки `home` в Linux). Относительный – **часть пути**, если следовать от расположения скрипта Python.

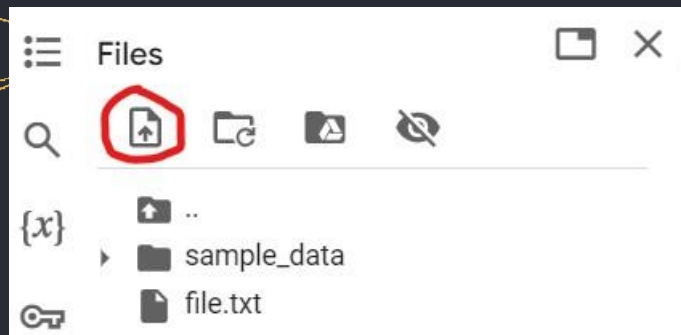
```
fin = open(r'E:\Python\data\text.txt')
```

```
fin = open(r'data\text.txt')
```

# Открытие файлов в Google



Самый простой способ – использовать **вкладку «файлы»**.



При нажатии на иконку файла открывается **файловый менеджер**, в котором мы можем выбрать файл из нашего компьютера.

Теперь мы сможем обращаться к этому файлу с помощью **относительных путей в Colab**.

# Режимы open()

Самые распространенные:

`'r'` – чтение

`'w'` – запись

`'a'` – запись с дополнением

```
fin = open("file.txt", "r")
read_text = fin.read()
fin.close()

fout = open("out.txt", "w")
fout.write(read_text)
fout.close()
```

`'r'` – по умолчанию. Если такого пути не существует, возникнет **ошибка**.

`'w'` принимает **существующий** путь с **несуществующим** файлом. Если такой файл существует, то он будет **перезаписан**. Расширение может быть любым, но внутри файл будет **текстовым**.

Режим `'a'` упрощает запись в существующий файл. Он не будет перезаписан – новые данные **добавятся в конец**.

## Параметр encoding=

```
fin = open('file.txt', 'r')  
print(fin.read())
```

745A L 1K;8 :@0:>701@K

```
fin = open('file.txt', 'r', encoding='utf-16be')  
print(fin.read())
```

здесь были кракозябры

Рекомендуется всегда указывать кодировку и при чтении, и при записи. Если этого не делать, то есть шанс получить или записать кракозябры. Лучше всего работать с utf-8.

(Этот файл изначально был записан в кодировке utf-16be, поэтому открывать его нужно точно так же).

# Чтение файла

| Метод                                                | Пояснение                                                                                                                                         |
|------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>file.read()</code>                             | Прочитает файл <b>целиком</b> и вернет его большой строкой, где каждую строчку разделяет <code>\n</code> . Не лучшее решение для огромных файлов. |
| <code>file.readlines()</code>                        | Прочитает файл <b>целиком</b> и вернет <b>список строк</b> . В конце каждой строки будет прилеплен <code>\n</code> .                              |
| <code>file.readline()</code>                         | Считывает <b>одну строчку</b> и возвращает <b>строку</b> . В конце строки также будет <code>\n</code> .                                           |
| <code>for line in file:</code><br><code>.....</code> | Считывает <b>по одной строке</b> из файла <b>на каждой итерации</b> , пока не дойдет до конца файла. Лучший способ для огромных файлов.           |

# Запись файла

| Метод                                                                            | Пояснение                                                                                                                                                                                               |
|----------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>file.write(str)</code>                                                     | Записывает <b>строку</b> любого размера в файл. <b>Не ставит</b> <code>\n</code> в ее конце. Возвращает количество записанных символов.                                                                 |
| <code>file.writelines(list)</code>                                               | Записывает <b>список строк</b> в файл. Тоже <b>не ставит</b> <code>\n</code> в конце каждой строки.                                                                                                     |
| <code>file = open(path, 'w')</code><br><code>print(<b>any</b>, file=file)</code> | Записывает <b>любой тип данных</b> (в виде строки) в файл (нужно передать в параметр <code>file</code> результат <code>open()</code> ). Поддерживает f-строки и другие возможности <code>print</code> . |

# Заккрытие файла

Заккрытие файла **обязательно**:

информация запишется только **после закрытия**, а сам файл в этом случае наконец **исчезнет** из оперативной памяти.

```
fout = open('file.txt', 'w', encoding='utf8')  
fout.write('some text')  
fout.close()
```

# Контекстный менеджер

Пожалуй, самый удобный способ работы с файлами. В этом случае нам не нужно помнить о закрытии файла: когда мы уйдем с отступа, файл закроется сам.

```
with open('file.txt', 'w', encoding='utf8') as fout:  
    fout.write('some text')  
print('Файл записался и закрылся.')
```

# Пример

Итерируемся по строкам в файле:

```
with open('file.txt', 'r', encoding='utf8') as fin:  
    for line in fin:  
        print(line.rstrip())
```

три  
строки  
читаем  
в цикле

# Функции встроенного в os модуля path

| Функция                                     | Пояснение                                                                                                                                                                        |
|---------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>os.path.exists(path)</code>           | Проверяет, <b>существует ли</b> такой путь. Возвращает bool.                                                                                                                     |
| <code>os.path.isdir(path)</code>            | Проверяет, является ли <b>путь папкой</b> .                                                                                                                                      |
| <code>os.path.isfile(path)</code>           | Проверяет, является ли <b>путь файлом</b> .                                                                                                                                      |
| <code>os.path.isabs(path)</code>            | Проверяет, является ли <b>путь абсолютным</b> .                                                                                                                                  |
| <code>os.path.abspath(path)</code>          | Возвращает <b>абсолютный</b> путь по <b>относительному</b> .                                                                                                                     |
| <code>os.path.splitext(path)</code>         | Делит путь на <b>расширение</b> и <b>все, что ему предшествует</b> . Возвращает кортеж (путь, расширение) – например, ('E:/data/text', '.txt').                                  |
| <code>os.path.split(path)</code>            | Делит путь на <b>файл</b> и <b>все, что ему предшествует</b> . Возвращает кортеж (путь, файл) – например, ('E:/data/', 'text.txt').                                              |
| <code>os.path.join(path1, path2,...)</code> | <b>Объединяет</b> несколько строк в <b>один путь</b> . Не умеет работать с буквой диска (т.е., не получится правильно объединить 'C:' и 'Python'). Учитывает особенности системы |

# Обход папки в pathlib

`Path.glob('*.*txt')`, `Path.rglob('*.*txt')`, `Path.iterdir()` — обход папок

```
from pathlib import Path

for path in Path('data').glob('*.*txt'):
    print(path.name, path.stat().st_size)
```

**glob** — по маске в одной папке, **rglob** — рекурсивно по всем вложенным.

```
p = Path('data') / 'text.txt'
p.exists() # True/False
p.is_file() # файл?
p.is_dir() # папка?
p.suffix # '.txt'
p.stem # 'text'
p.parent # Path('data')
p.resolve() # абсолютный путь
```

pathlib удобнее os.path: пути — объекты, а не строки.

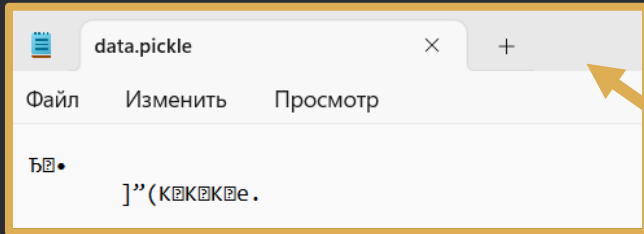
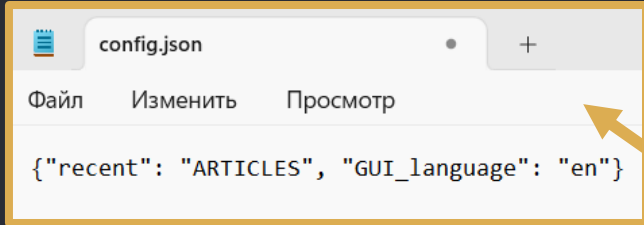
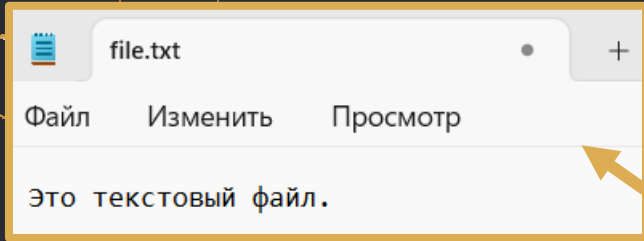
# Форматы файлов, сериализация

Файлы, которые умеет обрабатывать питон, находятся на своего рода **шкале по уровню читаемости человеком**:

1. **.txt файлы** - легко читаются человеком, трудно читаются скриптами (потому что не структурированы).

2. **.json, .csv файлы** - могут читаться как человеком, так и машиной.

3. **бинарные файлы** - не предназначены для чтения человеком.



# JSON

```
import json

dct = {'a': 1, 'b': 2}
with open('json_file.json', 'w',
          encoding='utf8') as fout:
    json.dump(dct, fout)
with open('json_file.json', 'r',
          encoding='utf8') as fin:
    js = json.load(fin)
type(js)
```

dict

json - JavaScript Object Notation; первоначально создавался для ЯП JavaScript, но может быть использован и для типов Python.

Это такой формат, в котором объекты Python записываются в машиночитаемом виде, но при этом могут читаться и человеком.

Запись данных в машиночитаемом виде называется сериализацией: когда мы считываем такие файлы снова программой, не нужно их специально парсить.

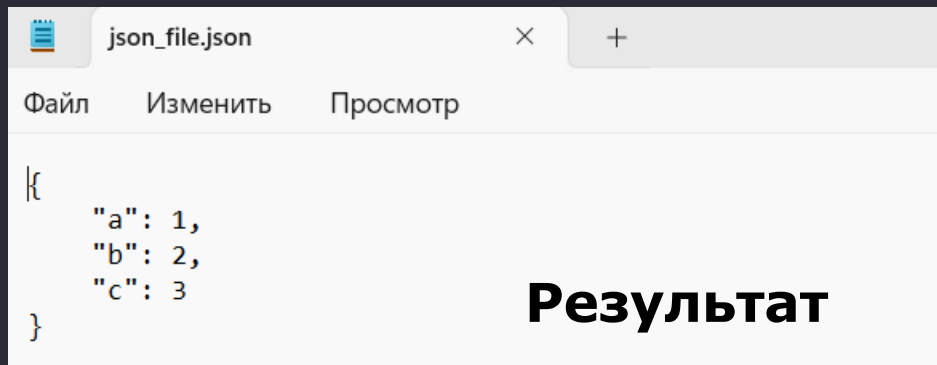
С json-файлами работает встроенный модуль json.

# Функции json

| Функция                                                            | Пояснение                                          |
|--------------------------------------------------------------------|----------------------------------------------------|
| <code>json.load(file)</code>                                       | Принимает файл. Загружает объект в скрипт Python.  |
| <code>json.dump(object, file, ensure_ascii=False, indent=4)</code> | Выгружает объект в файл.                           |
| <code>json.loads(string)</code>                                    | Преобразует строку в формате JSON в объект Python. |
| <code>json.dumps(object)</code>                                    | Преобразует объект в строку Python.                |

# Пример сериализации json

```
with open('json_file.json', 'r', encoding='utf8') as fin:
    dct = json.load(fin)
dct['c'] = 3
with open('json_file.json', 'w', encoding='utf8') as fout:
    json.dump(dct, fout, ensure_ascii=False, indent=4)
```



The screenshot shows a code editor window titled 'json\_file.json'. The editor has a menu bar with 'Файл', 'Изменить', and 'Просмотр'. The main area displays the following JSON text:

```
{
    "a": 1,
    "b": 2,
    "c": 3
}
```

Below the code, the word 'Результат' is written in a large, bold, black font.

Здесь мы десериализуем словарь вида `{'a': 1, 'b': 2}`, в коде добавляем элемент `'c': 3` и сериализуем его.

# Pickle

```
import pickle

lst = [1, 2, 3]
# Выгружаем
with open('list.pickle', 'wb') as fout:
    pickle.dump(lst, fout)

# Загружаем
with open('list.pickle', 'rb') as fin:
    pickled_lst = pickle.load(fin)
pickled_lst

[1, 2, 3]
```

Быстрее и лучше всего машина читает **бинарные файлы**, записывать которые умеет встроенный модуль **pickle**.

**Pickle** позволяет выгрузить **любой объект Python в бинарном виде**, а затем в любой момент **загрузить** его.

Также можно использовать библиотеку **dill** (ее надо установить `pip install dill`), у обоих модулей примерно одинаковый синтаксис.

# Спасибо за внимание

Контакты для связи: [ajumo@yandex.ru](mailto:ajumo@yandex.ru), [abyssian03@gmail.com](mailto:abyssian03@gmail.com)

GitHub: <https://github.com/abyssian03/Stud-2>